



Algoritma dan Pemrograman Komputer 1

Pertemuan 7: Fungsi/Method

M. Ma'ruf Qomaruddin Kafi – 5002241095
Thalita Ayu Antoniasari – 5002251011
Addien Zakia Hidayat – 5002231063
Layla Amrina Rosyada – 5002251101

12 November 2026

Departemen Matematika
Fakultas Sains dan Analitika Data
Institut Teknologi Sepuluh Nopember

Daftar Isi

Pengenalan

Dasar Penulisan Program Java

Pengenalan Fungsi/Method

Struktur Fungsi/Method

Aturan Penamaan

Jenis-jenis dan Contoh Method

Parameter vs Argument

Method Overloading

Studi Kasus

Best Practices

Latihan

Kesimpulan

Referensi

Pengenalan

Pengenalan

Raka, developer magang di sebuah toko online. Ia sudah menguasai *looping* dan *branching statement* (`break`, `continue`, `return`), sehingga program pengolah pesanannya sudah berjalan dengan baik.

Masalah Pertama
Semakin banyak fitur yang ditambahkan, semua kode ditulis di dalam `main`. Kini `main` milik Raka sudah **ratusan baris**, sulit dibaca, dan Raka sendiri bingung mencari bagian yang ingin diperbaiki.

Tugas
Hitung total belanja (harga $\times$ jumlah $+$ pajak 10%) di **5 tempat** berbeda pada program. Cara Raka: *copy-paste* rumus yang sama.

```
double total1 = harga1 * jumlah1 * 1.10;
double total2 = harga2 * jumlah2 * 1.10;
double total3 = harga3 * jumlah3 * 1.10;
double total4 = harga4 * jumlah4 * 1.10;
double total5 = harga5 * jumlah5 * 1.10;
```

Programnya jalan dengan benar. Raka merasa tidak ada masalah

Kejadian

Pajak naik menjadi 11%. Raka harus mengubah `1.10` menjadi `1.11` di **5 tempat**. Karena kurang teliti, satu tempat terlewat.

```
double total1 = harga1 * jumlah1 * 1.11;
double total2 = harga2 * jumlah2 * 1.11;
double total3 = harga3 * jumlah3 * 1.11;
double total4 = harga4 * jumlah4 * 1.10;   // lupa diubah!
double total5 = harga5 * jumlah5 * 1.11;
```

Akibatnya

Satu pelanggan ditagih dengan nominal yang salah, dan Raka harus mencari kesalahan di antara ratusan baris kode.

Kata Mentor
"Rumus yang sama jangan ditulis berulang. Tulis **sekali**, *beri* **nama**, *lalu* **panggil** *kapan saja dibutuhkan."*

```
1  public class TokoOnline {
2      public static double hitungTotal(double harga, int jumlah) {
3          return harga * jumlah * 1.11;   // pajak cukup diubah di sini
4      }
5
6      public static void main(String[] args) {
7          double total1 = hitungTotal(10000, 2);
8          double total2 = hitungTotal(25000, 1);
9          double total3 = hitungTotal(5000, 10);
10         double total4 = hitungTotal(15000, 3);
11         double total5 = hitungTotal(8000, 4);
12     }
13 }
```

Sekarang ketika pajak berubah lagi? Cukup ubah **satu tempat**. Inilah yang disebut **fungsi/method**.

Tentang `return`?
Pada materi sebelumnya, `return;` dipakai untuk **menghentikan program**. Di dalam method, `return` punya peran tambahan: **mengembalikan hasil** ke pemanggilnya. Raka memakainya untuk mencari nomor pesanan pertama yang mendapat hadiah (kelipatan 7), tanpa variabel penanda dan tanpa `break`.

```
1 public static int cariPesananHadiah(int awal, int akhir) {
2     for (int no = awal; no <= akhir; no++) {
3         if (no % 7 == 0) {
4             return no;      // ketemu, langsung kirim hasilnya
5         }
6     }
7     return -1;              // tidak ditemukan
8 }
```

Pengenalan

```java
public class PesananHadiah {
    public static int cariPesananHadiah(int awal, int akhir) {
        for (int no = awal; no <= akhir; no++) {
            if (no % 7 == 0) {
                return no;      // ketemu, langsung kirim hasilnya
            }
        }
        return -1;              // tidak ditemukan
    }

    public static void main(String[] args) {
        System.out.println(cariPesananHadiah(1002, 1020));
        System.out.println(cariPesananHadiah(1, 10));
        System.out.println(cariPesananHadiah(1009, 1013));
    }
}

// Output:
// 1008  (nomor pertama kelipatan 7 pada rentang 1002-1020)
// 7     (nomor pertama kelipatan 7 pada rentang 1-10)
// -1    (tidak ada kelipatan 7 pada rentang 1009-1013)
```

Hasil Akhir

- Method `main` Raka menjadi pendek dan rapi, tinggal memanggil method.
- Perubahan rumus cukup dilakukan di **satu tempat**.
- Setiap bagian program punya **nama** dan **tugas** yang jelas.

Pertanyaannya
Apa sebenarnya method itu, bagaimana cara menulisnya, dan apa saja jenisnya?

Dasar Penulisan Program Java

Kerangka Dasar Program Java

Sebelum mempelajari method lebih jauh, kita kenali dulu kerangka dasar sebuah program Java.

```java
public class Halo {
    public static void main(String[] args) {
        System.out.println("Halo, Dunia!");
    }
}

// Output:
// Halo, Dunia!
```

Program Java selalu terdiri dari **class**, dan di dalamnya terdapat **method** `main` sebagai titik awal program dijalankan.

Class

Definisi
Class adalah wadah yang membungkus seluruh kode program Java. Semua method harus berada di dalam sebuah class.

```
1 public class NamaClass {
2     // isi class (method-method) ditulis di sini
3 }
```

- Diawali kata kunci `class`, lalu nama class.
- Isi class dibungkus kurung kurawal { }.
- Konvensi: nama class diawali **huruf kapital** (contoh: `Halo`, `PencarianPesanan`).
- Nama file harus sama dengan nama class yang `public`, contoh `Halo.java`.

Method `main`

Definisi
Method `main` adalah **titik awal** program. Saat program dijalankan, Java mulai mengeksekusi dari method ini.

```
1 public static void main(String[] args) {
2     // statement program ditulis di sini
3 }
```

Aturan Menulis Method Baru

Benar

```java
public class Toko {
    public static void salam() {
        System.out.println("Halo!");
    }

    public static void main(String[] args) {
        salam();
    }
}

// Output:
// Halo!
```

Salah

```java
public class Toko {
    public static void main(String[] args) {
        public static void salam() {
            System.out.println("Halo!");
        }
        salam();
    }
}

// Compiler Error: method tidak
// boleh ditulis di dalam method lain
```

1. Method baru ditulis **di dalam class**, tetapi **di luar** method lain (termasuk `main`).
2. Letaknya bebas (di atas atau di bawah `main`), program tetap mulai dari `main`.
3. Method baru hanya berjalan jika **dipanggil**, contoh: `salam();`

Alur Eksekusi Program

Program dijalankan dari method `main`, lalu statement dieksekusi **berurutan dari atas ke bawah**.

```java
public class Urutan {
    public static void main(String[] args) {
        System.out.println("Langkah 1");
        System.out.println("Langkah 2");
        System.out.println("Langkah 3");
    }
}
```

Output Program

```
Langkah 1
Langkah 2
Langkah 3
```

Sebuah class juga boleh berisi **lebih dari satu method**. Method selain `main` hanya berjalan ketika **dipanggil**. Inilah topik kita selanjutnya.

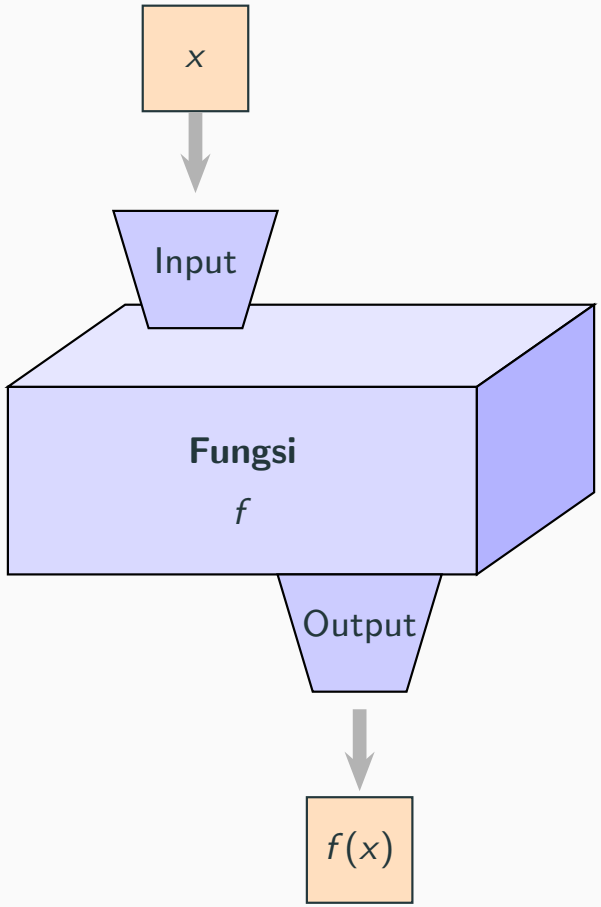
Pengenalan Fungsi/Method

Apa itu Fungsi/Method?

Definisi
Fungsi/Method adalah kumpulan beberapa pernyataan yang dikelompokkan untuk melakukan suatu operasi tertentu.

Analoginya, fungsi/method itu seperti fungsi $f(x)$ pada matematika yang bertugas untuk:

- Menerima input x (*parameter*).
- Memproses input tersebut f (*statement*).
- Menghasilkan output $f(x)$ (*return value*).



Tanpa vs Dengan Method

Tanpa Method

```java
public class TanpaMethod {
    public static void main(String[] args) {
        // Luas Persegi 1
        int p1 = 5, l1 = 3;
        int luas1 = p1 * l1;
        System.out.println("Luas 1: " + luas1);

        // Luas Persegi 2
        int p2 = 10, l2 = 4;
        int luas2 = p2 * l2;
        System.out.println("Luas 2: " + luas2);

        // Luas Persegi 3
        int p3 = 8, l3 = 2;
        int luas3 = p3 * l3;
        System.out.println("Luas 3: " + luas3);

        // Luas Persegi 4
        int p4 = 7, l4 = 5;
        int luas4 = p4 * l4;
        System.out.println("Luas 4: " + luas4);
    }
}
```

Dengan Method

```java
public class DenganMethod {
    public static int hitungLuas(int p, int l) {
        return p * l;
    }

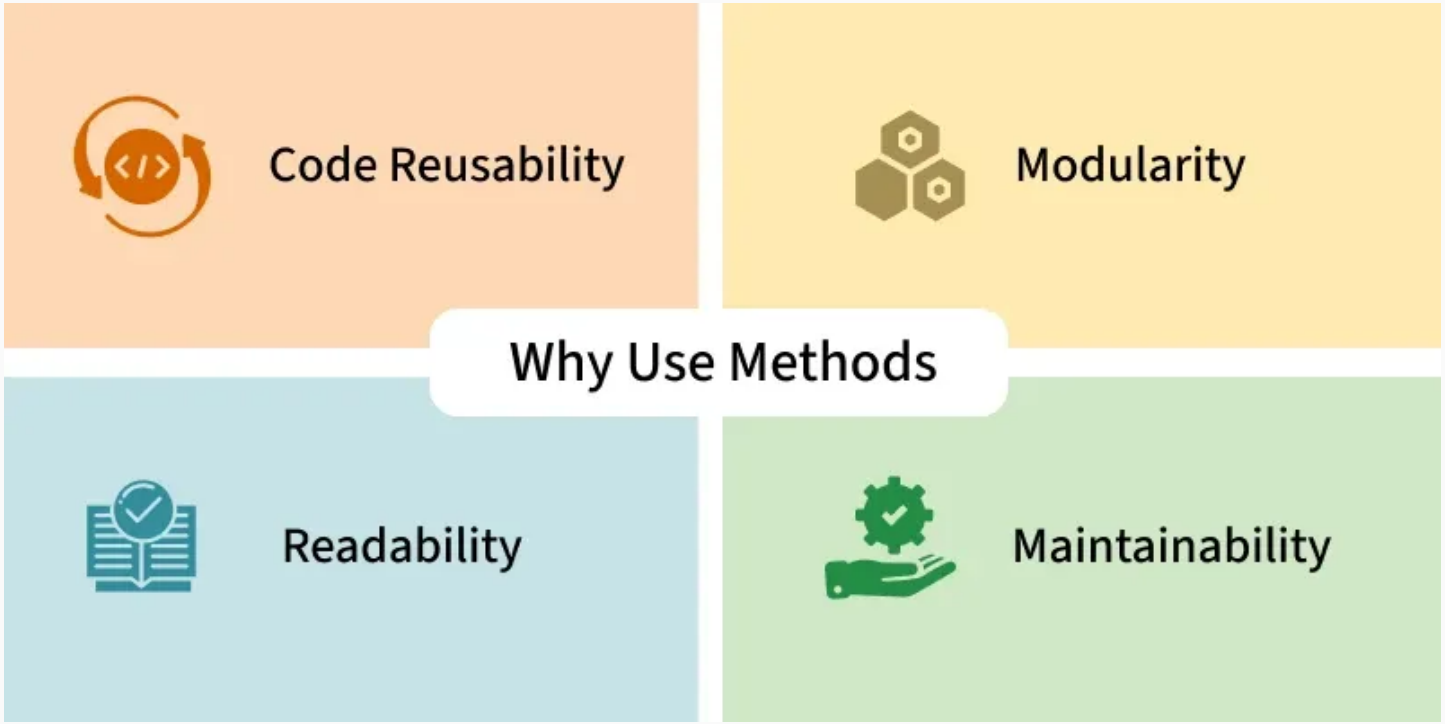
    public static void main(String[] args) {
        System.out.println("Luas 1: " +
            hitungLuas(5, 3));
        System.out.println("Luas 2: " +
            hitungLuas(10, 4));
        System.out.println("Luas 3: " +
            hitungLuas(8, 2));
        System.out.println("Luas 4: " +
            hitungLuas(7, 5));
    }
}
```

Quote of the Day

"Functions should do one thing. They should do it well. They should do it only."

– Robert C. Martin

Manfaat Penggunaan Method



Manfaat penggunaan method adalah **Code Reusability** (kode dapat digunakan berulang), **Modularity** (program dipecah menjadi bagian kecil), **Readability** (kode lebih mudah dibaca), dan **Maintainability** (kode lebih mudah dirawat).

Sumber: GeeksforGeeks (Methods in Java)

Manfaat Penggunaan Method

Keuntungan	Deskripsi
Reusability	Dapat dipakai berulang tanpa menulis ulang kode
Modularity	Memecah program kompleks menjadi bagian kecil
Readability	Kode lebih terstruktur dan mudah dibaca
Maintainability	Mudah dirawat
Testing	Mudah diuji secara terpisah
Collaboration	Memungkinkan kerja tim yang lebih baik

Tabel 1: Keuntungan Menggunakan Method dalam Pemrograman

Konsep Fundamental Method

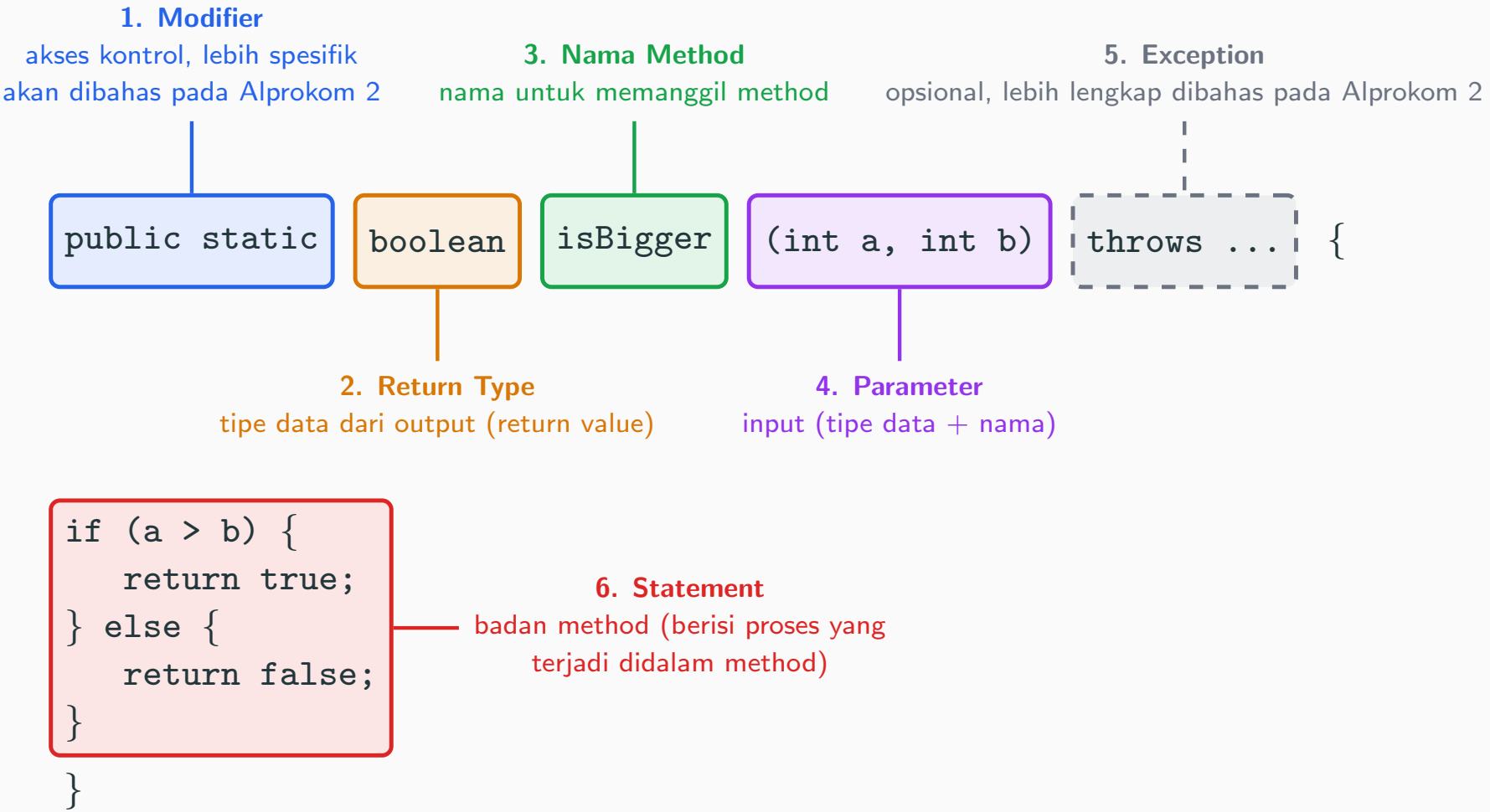
3 Fondasi Utama
Dalam memahami method, terdapat 3 konsep fundamental yang harus dikuasai:

1. **Parameter** – Input data yang diproses method (x).
2. **Statement** – Proses yang dilakukan di dalam method (f).
3. **Return Value** – Output hasil pemrosesan method ($f(x)$). Tipe datanya disebut *return type*.

Catatan Penting
Meskipun secara teknis parameter bisa kosong (tanpa input) dan return type bisa `void` (tanpa output), ketiga konsep ini merupakan fondasi pemahaman method.

Struktur Fungsi/Method

Struktur Fungsi/Method



Struktur Fungsi/Method

Contoh Struktur

```
1 public static boolean isBigger(int a, int b) {
2     if (a > b) {
3         return true;     // Return Value
4     } else {
5         return false;    // Return Value
6     }
7 }
```

Catatan

Komponen **Exception** tidak dipakai pada contoh ini.

Aturan Penamaan

Aturan Wajib Penamaan

Aturan wajib penamaan dalam Java adalah urusan dengan **Compiler**. Jika dilanggar, program **tidak akan bisa di-compile**.

Aturan Dasar:

1. Tidak boleh mengandung spasi.
2. Harus diawali huruf, tanda $, atau garis bawah _.
3. Tidak boleh menggunakan kata kunci (keyword) Java seperti `int`, `class`, `public`, dll.
4. Bersifat *case-sensitive* (huruf besar dan kecil dibedakan).

Contoh Pelanggaran
Menulis nama method dengan spasi:

```
public static void hitung luas() { ...}
```

Hasil: Compiler Error (Illegal start of expression).

Konvensi/Kesepakatan Penamaan

Konvensi hanyalah **kesepakatan**. Jika dilanggar, program **tetap berjalan**, tapi kode menjadi tidak rapi dan menyulitkan orang lain yang ingin membaca.

Kesepakatan	Contoh
`camelCase`	`calculateArea(), getUserName()`
Awalan huruf kecil	`make(), processData(), isEmpty()`
Kata kerja deskriptif	`convertToMeter(), validateInput()`
Konsisten	`getNama(), setNama(), isNama(), hasNama()`

Kenapa Harus Diikuti?

- *Clean code* (kode lebih mudah dibaca).
- *Clean code* sering masuk dalam kriteria penilaian praktikum.

Jenis-jenis dan Contoh Method

Method Tanpa Parameter

Contoh Method Tanpa Parameter

```
1  class MethodSederhana {
2      public static void sapa() {
3          System.out.println("Halo! Selamat belajar Java!");
4      }
5
6      public static void main(String[] args) {
7          sapa();  // Memanggil method
8          sapa();  // Bisa dipanggil berkali-kali
9      }
10 }
```

Output Program

```
Halo!  Selamat belajar Java!

Halo!  Selamat belajar Java!
```

Method Dengan Parameter

Contoh Method Dengan Parameter

```java
class MethodParameter {
    public static void sapaNama(String nama) {
        System.out.println("Halo, " + nama + "!");
    }

    public static void hitungLuas(int panjang, int lebar) {
        int luas = panjang * lebar;
        System.out.println("Luas: " + luas);
    }

    public static void main(String[] args) {
        sapaNama("Budi");
        hitungLuas(5, 3);
    }
}
```

Output Program

```
Halo, Budi!
Luas:  15
```

Method Dengan Return Value

Contoh Method Dengan Return Value

```java
public class MethodReturn {
    public static int tambah(int a, int b) {
        return a + b;
    }

    public static boolean adalahGenap(int angka) {
        return angka % 2 == 0;
    }

    public static void main(String[] args) {
        int hasil = tambah(10, 5);
        boolean cek = adalahGenap(hasil);

        System.out.println("10 + 5 = " + hasil);
        System.out.println(hasil + " genap? " + cek);
    }
}
```

Output Program

```
10 + 5 = 15
15 genap?  false
```

Contoh Method Rekursif (Faktorial)

```
1  class MethodRekursif {
2      public static int faktorial(int n) {
3          if (n == 0 || n == 1) {
4              return 1;
5          } else {
6              return n * faktorial(n - 1);
7          }
8      }
9
10     public static void main(String[] args) {
11         int angka = 5;
12         int hasil = faktorial(angka);
13         System.out.println(angka + "! = " + hasil);
14     }
15 }
```

Output Program

```
5!  = 120
```

Parameter vs Argument

Contoh Perbedaan Parameter dan Argument

```java
public class ContohParameterArgument {

    // 'nama' dan 'umur' adalah Parameter
    public static void perkenalan(String nama, int umur) {
        System.out.println("Nama: " + nama + ", Umur: " + umur);
    }

    public static void main(String[] args) {
        // "Alice" dan 20 adalah Argument
        perkenalan("Alice", 20);
    }
}
```

Parameter	Argument
Variabel dalam deklarasi method	Nilai asli yang diberikan saat pemanggilan
`String nama` dalam deklarasi	`"Alice"` saat memanggil method
Seperti variabel yang didapat	Seperti nilai yang diisi
Ditentukan saat menulis method	Diberikan saat menggunakan method

Tabel 2: Perbedaan Konseptual Parameter dan Argument

Method Overloading

Konsep Method Overloading

Apa itu Overloading?
Membuat beberapa method dengan **nama sama** tetapi **parameter berbeda** (jumlah parameter atau tipe data parameter berbeda).

Contoh Overloading

```
1  public class Calculator {
2      // Versi 1: dua parameter integer
3      public static int tambah(int a, int b) {
4          return a + b;
5      }
6
7      // Versi 2: tiga parameter integer
8      public static int tambah(int a, int b, int c) {
9          return a + b + c;
10     }
11
12     // Versi 3: parameter double
13     public static double tambah(double a, double b) {
14         return a + b;
15     }
16 }
```

Penggunaan Method Overloading

Implementasi dalam Program

```
1  public class Calculator {
2      public static void main(String[] args) {
3          // Memanggil versi berbeda berdasarkan parameter
4          int hasil1 = tambah(5, 3);
5          int hasil2 = tambah(1, 2, 3);
6          double hasil3 = tambah(2.5, 3.7);
7
8          System.out.println("5 + 3 = " + hasil1);
9          System.out.println("1 + 2 + 3 = " + hasil2);
10         System.out.println("2.5 + 3.7 = " + hasil3);
11     }
12 }
```

Output Program

```
5 + 3 = 8

1 + 2 + 3 = 6

2.5 + 3.7 = 6.2
```

Studi Kasus

Studi Kasus 1: Kalkulator

Implementasi Kalkulator dengan Method

```java
public class Kalkulator {
    public static double tambah(double a, double b) {
        return a + b;
    }

    public static double kurang(double a, double b) {
        return a - b;
    }

    public static double kali(double a, double b) {
        return a * b;
    }

    public static double bagi(double a, double b) {
        if (b == 0) {
            System.out.println("Error: Pembagian dengan nol!");
            return 0;
        }
        return a / b;
    }

    public static void main(String[] args) {
        System.out.println("10 + 5 = " + tambah(10, 5));
        System.out.println("10 - 5 = " + kurang(10, 5));
        System.out.println("10 * 5 = " + kali(10, 5));
        System.out.println("10 / 5 = " + bagi(10, 5));
    }
}
```

Studi Kasus 2: Validasi Input

Implementasi Validasi Input dengan Method

```java
public class Validator {
    public static boolean isValidEmail(String email) {
        return email.contains("@") && email.contains(".");
    }

    public static boolean isValidAge(int age) {
        return age >= 0 && age <= 150;
    }

    public static boolean isStrongPassword(String password) {
        return password.length() >= 8 &&
               !password.equals(password.toLowerCase());
    }

    public static void main(String[] args) {
        System.out.println("Email valid? " + isValidEmail("user@example.com"));
        System.out.println("Umur valid? " + isValidAge(25));
        System.out.println("Password kuat? " + isStrongPassword("Pass1234"));
    }
}
```

Best Practices

Best Practices Penggunaan Method

Tips dan Rekomendasi

- **Satu tugas satu method** - Setiap method harus melakukan satu tugas spesifik
- **Nama deskriptif** - Nama method harus jelas menjelaskan fungsinya
- **Parameter minimal** - Gunakan parameter secukupnya, hindari terlalu banyak
- **Return early** - Keluar dari method secepat mungkin jika kondisi terpenuhi
- **Fokus pada tugas** - Method sebaiknya hanya mengolah input menjadi output
- **Dokumentasi** - Berikan komentar untuk method yang kompleks

Latihan

Latihan 1: Menghitung Volume Tabung
Buatlah sebuah method bernama `hitungVolumeTabung(double r, double t)` yang menerima jari-jari (`r`) dan tinggi (`t`), lalu mengembalikan nilai volume tabung.

Rumus: $V = \pi \times r^2 \times t$

Petunjuk: Gunakan `Math.PI` untuk nilai π.

Contoh Output

```
Masukkan jari-jari:  7
Masukkan tinggi:  10
Volume Tabung:  1539.38
```

Latihan 2: Cek Bilangan Prima

Buatlah method bertipe boolean bernama `isBilanganPrima(int n)`. Method ini harus mengembalikan nilai `true` jika bilangan tersebut adalah bilangan prima, dan `false` jika bukan. Bilangan prima adalah bilangan yang hanya habis dibagi 1 dan dirinya sendiri (2, 3, 5, 7, 11, ...).

Contoh Output

```
Masukkan bilangan:  7
Apakah 7 bilangan prima?  true
Masukkan bilangan:  9
Apakah 9 bilangan prima?  false
```

Latihan 3: Deret Fibonacci
Buatlah method rekursif `fibonacci(int n)` yang mengembalikan nilai suku ke-n dari deret Fibonacci.

Rumus Fibonacci:

- F(1) = 1, F(2) = 1
- F(n) = F(n-1) + F(n-2) untuk $n > 2$

Contoh Output

```
Masukkan suku ke-n:  7
Nilai suku ke-7 adalah:  13
```

Kesimpulan

Kesimpulan

Inti Bab 9: Fungsi/Method

- **Fungsi/Method** adalah blok kode yang melakukan tugas spesifik dan dapat dipanggil berulang
- **Struktur method** terdiri dari 6 komponen: modifier, return type, nama method, parameter, exception, statement
- **Jenis method**: tanpa parameter, dengan parameter, dengan return value, dan rekursif
- **Method overloading** memungkinkan beberapa method dengan nama sama tetapi parameter berbeda
- Penggunaan method meningkatkan **Reusability, Modularity, Readability, dan Maintainability** kode

Referensi

Referensi:

- **Modul Praktikum Algoritma dan Pemrograman - Modul 9**
  Departemen Matematika FSAD ITS
- **Oracle Java Tutorials - Defining Methods**
  `https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html`
- **GeeksforGeeks - Methods in Java**
  `https://www.geeksforgeeks.org/methods-in-java/`
- **W3Schools - Java Methods**
  `https://www.w3schools.com/java/java_methods.asp`
- **Programiz - Java Methods**
  `https://www.programiz.com/java-programming/methods`

Terima Kasih